ActInf GuestStream 116.1 ~ Max Aifer: "Solving the compute crisis with ph

GuestStream | Max Aifer | 1:11:03

Hello, welcome everyone. It is August 6th, 2025. We're live in active guest stream number 116.1
with returning guest Max Afer, who will be discussing this paper,
solving the compute crisis with physics-based basics. Max will go over the paper, talk about a
few different things, then I will check in the live chat and read any questions that people have.
So looking forward to people's questions. And Max, thank you again for joining. Looking forward to
the presentation. All right. Thank you for the introduction. Yes, I'm Max Afer, and I work with
normal computing. And as Daniel mentioned, this is my second time on the stream with the Active
Inference Institute, the first time I was talking about a paper called Thermodynamic Bayesian Inference,
which is an example of one of our algorithms we've been developing in thermodynamic computing.
And this time I'm going to be talking about this new perspective article we've written called
Solving the Compute Crisis with Physics-Based ASICs. And just a little word about the background on this.
This kind of emerged out of an industry workshop that Normal Computing, the company I work at,
we hosted at our office in New York, because we had been focusing up to that point on, and still are,
on the ideas of thermodynamic computing and applying those to machine learning, and in particular,
you know, probabilistic machine learning. But we noticed that there's a lot of other people,
both in industry and academia, working on different approaches to unconventional computing. And we
figured, well, there must be something that we can learn from each other. So we hosted this workshop.
And it turned out, I think, that a kind of more, a larger, like unifying framework started to emerge,
that there's a bigger picture of something that these different approaches have in common. Even though,
even though when we move beyond the standard digital paradigm we're familiar with, we get kind of a zoo of
strange, exotic new components and circuits that we're using, there is still some kind of pattern or structure
that we can identify and categorize to think about these things. And so that's kind of where this paper
came out of, that us and the other people who were at that workshop thought it was worth writing a paper on.
And so I'm going to, after I kind of finish, you know, giving a little bit of the background and overview,
I'll kind of scroll through the paper and give more detailed explanations and commentary on it.
And also, of course, I encourage, you know, anyone to go and check it out for yourself.
So this is a perspective article and we're not going to be presenting new, like, theoretical results in here,
per se, or simulation results. But it's more our view of the connections between these different
computing paradigms and where we see the field going. But in a nutshell, I guess it's the reason why I
think we see so many of these new exotic computing paradigms starting to take off now is because
there's kind of a new challenge for computing, which is the amount of compute that's required for AI is
just a very large amount of compute and it requires a very large amount of energy. And that's not, you know,
not, I guess, a new challenge for computing in a way and that there have always been new algorithms
that demanded more compute. But there is something different about it, which I'll get into when I talk
about the end of Denard scaling in that we'll have to solve it in a different way than it was solved

before. And so I guess that at a high level, you can kind of think of it as instead of just trying to make chips bigger or cram more transistors onto a chip. People are now looking into ways of using a different kind of fundamental structure for computing or replacing, you know, using something else instead of a transistor to do computing logic gates or storing information or taking those transistors we have and using them in a different way, modeling their behavior in a more complicated way. And of course, there's also ways we can rethink the architecture of a chip, something like a GPU or a CPU. There are a lot of kind of things in common in the circuit architecture from one to another, and that hasn't been changed very much in a long time, at least in, you know, big commercial platforms. And so people are also thinking of new kind of novel architectures. And the kind of family of approaches I'm going to be talking about are what we call physics-based ASICs, which we've kind of introduced this term and given a bit of a definition in this paper. But what you can think of it as is saying instead of trying to abstract the physics away and treating my circuit as something that behaves like an ideal model of computation, like a Turing machine or finite state machine, I'm going to actually model the physics of the components of a circuit and use those in an algorithm somehow. So I'm going to use, you know, physical components on my chip and pay attention to what kind of physical processes they undergo and trying to exploit those to do some kind of useful computation.

So and then, of course, we can get into like the idea of hardware algorithm co-design, where we kind of look at two different things. We look at like what are the kinds of physical processes that this device can do? And then on the other hand, we look at what kinds of processes are useful for my algorithm or what kind of primitive operations are needed by the algorithm I'm trying to run. And we try to kind of find the overlap between those two categories, basically. And that's basically what a lot of us are working on more or less right now, including at normal computing, is this approach to hardware algorithm co-design. So that's kind of the high level overview, I guess, of the ideas in the paper.

And so now I guess I'll go into a little bit more detail. So I had mentioned earlier that the problem of having applications, algorithms that require a lot of compute is not a new problem per se. But the thing that is different is that we are now kind of firmly in the like post-Denard scaling era, and arguably in the post-Mohr's Law era. So I guess I'll say a little bit about what that means.

So what Denard scaling was about was that for a long time, you know, decades up until the early 2000s, when people made new microprocessors, made new CPUs and I guess GPUs towards the end of that time period,

they would shrink the transistors, you know, they'd make the transistors smaller so they could fit more transistors on a chip, right? And this is how Moore's Law, which is maybe a little bit more, more people are a little bit familiar with Moore's Law than Denard scaling. But both were driven by that trend, that people would make transistors smaller, and so they could fit more transistors on a chip of the same size. But what Denard noticed is that when you make a transistor smaller, that also changes the amount of power that you need to operate it, because the supply voltage is also reduced by the same factor when you make that transistor smaller. So this was really great, right? You know, in engineering, it's not so often you get something that's like a kind of a free lunch, but this was maybe the

closest thing you could get for a free to a free lunch is that we make the transistors smaller, and not only can we have more on the chip, but they consume less power, they are also going to dissipate less heat, so that helps us out with cooling, right? And one way of kind of summarizing that, that idea is that independent of how small your transistors are, in the Denard scaling era, the density of power consumed per unit area was constant. So in other words, if I have a chip that's, you know, a centimeter by a centimeter, doesn't matter how many transistors are on it, a million, a billion, or a trillion, or whatever, we would assume that they would all consume the same amount of power. And so that was, in a nutshell, like how the challenges of more and more demanding applications was met historically, was that we would make the transistor smaller, put more on the chip, and we would also have better energy efficiency because of Denard scaling. But the reason that, so I guess, yeah, Denard scaling ended in like around, you know, people will say different years, maybe 2005, something around there, maybe a little earlier. But the reason it ended is that there are actually not every, not every electrical, electrical property of the transistor scales in the same way when you make it smaller. And in particular, it was without going, you know, too deep into detail here, it ended up being, turning out that the leakage current and the threshold voltage don't scale in that way. And so we could only kind of make them so small and keep reducing the power consumption. And then so once transistors were small enough that the leakage current became, you know, very important, that that trend ended. And so after that, you know, we could still make transistors smaller to some extent, but that wouldn't really change the power, or they would still consume, you know, roughly the same amount of power. And so that started to create problems for cooling, right? Because we can cram more and more transistors in a small area. But that means the power, the density of power per unit area is now going to start increasing after the end of Denard scaling. And so people would do things like say, we're only going to turn on part of the chip at a time, because otherwise, there would just be too much heat, stuff like that. And that was really what I think kicked off kind of like the multi core era, where people then started to say, well, because we can't really get much more mileage by just making the transistor smaller and putting more on the on the chip, what we're going to start doing is, well, we're gonna have multiple processors, basically. And so when we need more compute, we'll just get more processors, we'll get more chips, and then we'll try to parallelize our algorithms, we'll try to make our algorithms into, we'll try to solve our problem using an algorithm that can efficiently, effectively be parallelized across multiple chips that with a minimal amount of communication between them. And that basically brings us to where we are today, where, you know, the GPU, general purpose GPU is the dominant paradigm for, you know, some of the most important algorithms, particularly in AI now. So yeah, I guess that's kind of the starting point. And then the question is, well, what are the, you know, what are the problems with this GPU paradigm? Or is this, is this way of scaling things up, just getting more and more GPUs, going to work? Or if not,

you know, why are we considering these physics based ASICs instead? Well, so basically, the issue is that right now, the, because of this situation where we can't reduce power consumption just by making the transistor smaller, if you have, you know, twice as many GPUs, they're going to be consuming twice the amount of power. And so there's, you know, a huge increase in the amount of energy that's consumed due to like AI that's anticipated with people putting power plants in their data centers or data centers in their power plants, or however you want to look at it, just because you need a huge amount of energy to power, you know, training, particularly training, maybe even inference to some extent to us now. And then also, there's just the sheer cost of getting all that hardware, right? The cost of buying all the GPUs that you need to do a training run for a foundation model. And so that's why you have the, you know, the big companies like OpenAI, and etc, saying that they're literally, you know, melting, their GPUs are melting, because there's just too much throughput, basically, for them to handle. So that's why we're there. That's basically the summary of the problem that we're trying to solve, or that we're, we're talking about different solutions to that problem in this perspective article. Okay, so I think that's also a good point to transition into talking about the idea of hardware algorithm co-design. So I mentioned that, because the way that we're scaling up compute now is getting more chips, and we need to parallelize our algorithms, the dominant algorithms that we see in the model architectures that we see in AI, are ones that can be parallelized very effectively. And that's something that was pointed out in a well, you know, a well-known paper called the hardware lottery. And it was talking about how model architectures like transformers, and so on, which are powering, you know, the large language models, are not necessarily better than many other approaches, different approaches from a first principles, like algorithmic standpoint. But the reason why they're so successful, and these algorithms have kind of won out, is because they they are more suitable for the hardware paradigm, they parallelize better on GPUs. And so that's something that in many now have kind of come around to that view, that the transformer, the reason why it's so successful and prevalent, I guess, is not because it's better than, you know, other solutions, like recurrent neural networks, or LSDM, or whatever, but just because it parallelizes better on GPUs. Now, we can kind of take this line of thinking and generalize it a little bit further, in that, so clearly, it's not just the question of like, how good is the hardware, or how good is the algorithm, but the question of how suitable is the, are the algorithms and hardware for each other, is kind of equally important. So that's kind of maybe a slightly more general way of thinking about that idea of the hardware lottery. And if we think about, you know, how does it make sense for us to design better algorithms and hardware, from that perspective, I think there's a good case to be made that we don't want to treat it as two separate optimization problems of either, on the one hand, we'll consider the algorithm fixed, and we'll try to make the hardware as fast as possible for running that algorithm. Or on the other hand, we'll consider the hardware fixed, and we'll design the algorithm to run as fast as possible on that hardware. But instead of either of those, we approach it as a joint optimization problem, as how do we design the best combination of hardware and an algorithm to solve this problem, you know, as efficiently as possible. And so that approach is hardware algorithm

co-design. And in many of, you know, the ideas in this paper on physics-based ASICs are taking a co-design approach like that, I guess.

So I guess that with that said, I'll start to jump into like, what is a physics-based ASIC?

And how do we even define that? So I'll start out with kind of an observation about why we think that using the physics of devices can give us more, you know, more computational power, basically.

So we give an example in the Perspective article of the problem of simulating the behavior of a single transistor, right? So you would think of it as, well, if I have, you know, software like SPICE or Cadence or something, they have some models, behavior models of how a transistor behaves, or like, you know,

given the input signals, what are the output signals and so on. But to have like a really detailed physical model of a transistor that kind of totally explains all of its observable behavior, you would need to, well, one way to do it is you could solve a system of partial differential equations for the transistor with, you know, many variables, many degrees of freedom. And if you wanted to like run that simulation on a standard like digital computer, you would end up using, you know, billions of transistors at least or more to simulate this one transistor. And you'd be doing, you know, a huge number of, of flops, you know, floating point operations just for a single time step to simulate that one transistor. And so once we kind of realize that, we can say, well, if it turned out that the behavior of a single transistor, kind of the detailed physical behavior was actually useful for some application, then instead of running this very costly simulation, we could just use that single transistor as kind of its own, its own model of its own behavior, basically, instead of, you know, some simulation of it, basically. So that's kind of the intuition for why we think that like hidden in the physics of common, like, common, like circuit components, there's a lot of computation in there that's, that's hidden in there somewhere. But then the question is, well, is that computation useful? So then we need to say, well, what are the primitive operations that are useful for the algorithms that we want to run? Is there some way for us to map them on to the physics of, you know, circuit components? Okay, so now that I've got to kind of cover that intuition, I'll start to walk through more of a, more of a definition, I guess, or maybe as close as we come to a definition in the perspective article for physics based ASICs. And the way we think we're thinking about it is that, so first of all, all integrated circuits use physics to do computing, there's no, you know, there's no such thing as a, as a circuit that doesn't use physics, because that ultimately the physics of the circuit and Kirchhoff's laws, and etc, are going to govern the behavior. So what do we really mean when we say a circuit that is using physics to compute? Well, for one thing, in, in any kind of common, like run-of-the-mill integrated circuit, that you'll find, there are a lot of, well, they, they basically kind of abstract away from the physics, so that it's possible to treat the circuit as an idealized model of computation, like a finite state machine, or a Turing machine, or something like that. And in order to make that possible, so that you can kind of ignore the physics and design algorithms on a higher level of abstraction, we need to suppress a lot of kind of messy behaviors that you would see, that you would find in systems in, in nature, and, you know,

kind of generic, you know, dynamical systems in physics, but you don't really observe these behaviors in circuits that we've designed. And so here are some examples of those, those messy behaviors that we tried to design integrated circuits to suppress. So one is like statefulness. So in a, in an integrated circuit, there is a clear separation between memory and compute. You know, you have some transistors that, and, and capacitors, etc, that are responsible for storing bits of information, for storing data, and other ones that are in logic gates that are responsible for processing that data.

And now more recently, people are kind of starting to merge the two. And that's, I think, kind of starts to fall within the realm of physics based ASICs. But in the conventional paradigm, you have this clear separation between, you know, logical operations and memory. And what that means is that the, the circuits that are responsible for the logical operations are supposed to behave statelessly. And so as an example of that, let's say that I have like a not gate, and I input a zero, it outputs a one, I input a one, it outputs a zero. It's only, its output is only supposed to depend on the input it's getting at any given time. It's not supposed to, its output isn't supposed to depend on previous inputs that it's gotten. Or in other words, it's not supposed to be kind of maintaining some internal state that keep that keeps track of its history, basically, it's supposed to behave statelessly. So that's one way in which circuits are

designed to suppress kind of a common, like generic physical behavior, because most dynamical systems in nature, of

course, are stateful in the way that they behave will depend on on their history. So that's one example. Another one is

unidirectionality of the flow of information. So I'll take the example of a not gate, again, if I change the voltage at the input, the voltage at the output is supposed to change in response to that. But we're not supposed to have information going back the other way, it shouldn't, if I applied a voltage, change the voltage at the output, that shouldn't affect the voltage at the input. And that's designing logic gates and, you know,

components that satisfy this kind of unidirectionality makes the like engineering of circuits a lot easier, because you can treat logic gates as blocks in a feed forward network, where information propagates through one block and then through the next one. And you don't have to worry about like kind of unwanted feedback loops, where information, you know, just kind of spontaneously goes propagates back through to the beginning of the circuit. And of course, you can design circuits with feedback loops, and many circuits do have feedback loops, like the one illustrated here. But the point being, though, the feedback loop has to be built in explicitly. So here you have, you know, logic gates processing bits, a memory device, a flop is updated, and then a signal is taken back to the beginning by some some wire. And this is the way that, you know, feedback loops are created in digital circuits, because you're not supposed to have information shouldn't flow back through the same logic gate. Okay, so and then again, if you compare this to how like, physical systems in nature behave, they often don't have like unidirectional flow of information, they have bidirectional flow of information. So you think of, for example, Newton's law, you know, for every action, there's an equal and opposite reaction. If you were if you have two masses on a

spring, if mass A is connected to mass B, if if B is pulled in one direction by A, then A is, you know, pulled in the other direction by B. And analogously, if we think of like an analog circuit, where you have different capacitors coupled to each other in some, you know, resistive network or something, they'll both, you know, be sharing charge with each other, and will their voltages will will both be changing at the same time with information flowing in both directions. So okay, so unidirectionality was another example of how conventional ASICs suppress a kind of a generic physical behavior to make them conform to some like ideal model of computation. Another important one is determinism. So a circuit is given if you give the same input multiple times, it should give the same output each time. And of course, you can use, you know, deterministic circuit for randomized algorithms if you want. And that'll generally be done using a pseudo random number generator, which is, you know, still going to be implemented by the deterministic algorithm on a deterministic circuit. But the point being, the circuit itself is supposed to, you know, to a very good approximation behave in a deterministic way where you give it the same input in the same, you get the same output. And so that's another, you know, thing that is not, well, it's not perfect, it's not possible to realize that perfectly. It's only possible to realize it to a good approximation, because, of course, all systems in nature have fluctuations, they have noise, which is guaranteed by thermodynamics. And so in order to suppress those fluctuations, or at least make them or protect against them so that you don't have random bit flips all the time, that's always going to cost some amount of energy. And there's going to need to be some kind of safeguards for errors, correction, or whatever, built in to enable this idealized deterministic behavior. So that was a third example of a kind of a behavior that is suppressed in conventional ASICs. And then I think the last example we gave here was synchronization. So when we have, you know, some very large circuit on a chip that with many different, you know, many different parts all doing something at the same time, there is typically going to be like a global clock that is sending out a clock pulse signal to, you know, all different parts of the chip at the same time. And so all parts of the chip are synchronized with each other and with the centralized clock. Now, this is another kind of behavior that we don't often see in systems we find in nature. There are many systems that exhibit synchronization to some extent, but often synchronization happens in a distributed way. So, you know, or if you have something like a Kuramoto model, for example, which can be used to model how, I believe it's used to model how neurons synchronize with each other in the brain or things like that, often models of synchronization that are supposed to be realistic or ones that are supposed to model how synchronization actually happens in nature are using a kind of distributed model of synchronization where different, um, different kind of components will talk to each other, but they're not all talking to one centralized clock. Of course, there are exceptions to that. Uh, but that's probably the more like generic physical situation where you have, uh, if you have synchronization, it happens in a distributed way. And then often in arbitrary dynamical systems, you won't really have any synchronization at all. Um, so this was another behavior that where we're kind of building in something, uh, where we, we're going through some effort to give our system a property that is kind of uncommon to find in nature is, is

somewhat unnatural somehow. And so for all of these properties that allow us to treat integrated circuits as idealized models of computation, we're going to pay some costs in order to make them, to realize these properties approximately, uh, and they can generally, they can only be realized approximately, not perfectly. Uh, and in general, we're going to incur some additional costs in terms of dissipation, uh, you know, how much, yeah, how much energy is dissipated, um, and in terms of speed. So, uh, in order to make these approximately true, we're going to have to accept some limitation on how fast we can run our circuit. Uh, a good example probably being like synchronization. If you imagine that every part of the chip needs to be synchronized, that global clock pulse, well, it takes some amount of time for that clock pulse to reach all the different parts of the chip. So that could be a limitation on how fast you can make your clock cycles, right? Because you need to allow enough time, uh, for the clock pulse to reach all different parts of the chip. Um, similarly, determinism is going to incur some cost because we need to suppress noise or, or make our signals large enough that the noise is like negligible relative to the signals, which is a common, uh, way of, of making circuits deterministic. Um, and so this observation that these idealized properties all incur some cost in terms of energy or time or something like that, uh, generally energy and time, uh, because of that, we started to say, well, what if we design circuits without worrying about these properties or saying we're going to relax these requirements that they, they have these properties and then maybe our circuits will be more energy efficient
and maybe they'll run faster. Uh, but then we'll have to kind of think in a more complicated way when we design our algorithms because we can no longer, or when we, even when we design our circuit, both the circuits and the algorithms, because it's not as easy to, uh, for example, to compose operations, uh, because now instead of treating each component as a, a block, you know, element that, uh, feeds information for that, you know, has unidirectional information flow and is stateless and deterministic. Now we, each component is kind of a little bit more messy and we have these messy behaviors that we need to, uh, you know, deal with somehow when we design our algorithms. Um, so that's kind of, I guess, how I would think about like defining, um, what a physics-based ASIC is, is an ASIC that is designed, uh, which is not designed to satisfy all of these properties approximately, um, and, uh, violates, you know, at least one of them or in, in oftentimes more. Uh, and as a result, we hope to be able to, um, make circuits more energy efficient, uh, you know, and faster that way. Um, so now I've kind of given, you know, a model for, like, how to think about what a physics-based ASIC is. Um, I guess I can talk a little bit about like some examples to, uh, approaches here. Um, but actually, I guess before I go into the examples of hardware paradigms, maybe I'll pause and see if there were any, um, any questions that came up in, in the chat and if there were, I can, I can take some of those.
Um, I'll just read two helpful comments by your colleague Patrick Coles, who wrote,
joint optimization, optimizing the hardware and algorithms together can be more powerful than individually optimizing them independently. And in a sense, the more properties on this list that you assume, the more you move away from physics, what physics does naturally. Sorry about the audio, by the way. Oh, no worries. Great. Yeah. I thought those were great comments.

Thanks. Thanks for the comments, Patrick. Okay. Um, all right, great. Well, so I guess I'll, I'll keep, uh, you know, pressing forward and talk about a few examples of like kind of hardware paradigms that start to, um, abandon some of these properties, I guess, in, in various ways. Uh, so one of course, uh, is analog circuits. Um, here we have an example of like an RL or RC, uh, kind of, uh, uh, RC circuit. Um, and you could have many little RC circuits like this that are all coupled to each other. You know, in general that, you know, these can be noisy. Uh, they could have, you know, more or less noise, or they could even have extra noise sources included in them. Um, and when we have a bunch of these coupled together, what, how they behave is modeled by some kind of differential equation or some stochastic differential equation, uh, that describes how all the voltages and currents change the function of time. Uh, and so when we have a system like that, well, for one thing, it's not deterministic if there's, you know, thermal noise. Uh, for another, um, it, uh, has information that flows in both directions across all the couplings. Um, and it has components that are, that behave in a stateful way like these capacitors. Their voltages are, are actually playing kind of an algorithmically important role of storing state. Um, and so you have kind of somehow, you know, memory and processing happening in the same place. Um, I think, I wonder if there's any other properties that, that, that, well, I guess it was all four of them just with that, you know, the simple example of the, um, you know, stochastic analog, you know, coupled RC, uh, network. Um, and, and now there's other kinds of circuits that are, uh, you know, use, um, that do processing using, uh, kind of similar like networks, uh, like, like resistive crossbar arrays are, you know, can be used for matrix vector multiplication using, uh, you know, an array of, uh, of resistors. Uh, now I'm not sure if I would exactly consider this like analog computing, although maybe it is in some way in that you're, you're doing addition using Kirchhoff's, you know, current law, instead of using addition in binary, uh, using, um, a bunch of, uh, you know, a bunch of logic gates. Uh, and so we're somehow using kind of the analog, uh, current signals in order to do the addition of, uh, you know, of our numbers. Um, and so I guess, well, another example of, of circuits that violate determinism would be like, uh, stochastic magnetic tunnel junctions where you can have, um, stochastic switching, uh, which happens when the magnetization direction in one of the ferromagnetic layers, uh, can spontaneously switch, uh, the magnetization can switch direction as a result of thermal fluctuations. Um, and so now we have like non-determinism baked in at kind of the, the smallest level of, of the circuit instead of building our circuits out of, um, building blocks that are deterministic, we're building it out of building blocks that behave in a stochastic way. Uh, and that can be used for like stochastic computing, for example, or it can be used for like, for computing based on like P, uh, P bits or probabilistic bits. Uh, people commonly like use things like this for, um, like, uh, Bolson machines, uh, would be one example. Um, and then I guess, uh, uh, other kinds of like non-deterministic behavior in, in circuits would be like, uh, stochastic digital circuits, which can still be built without using any like more exotic components like MTJs, but just with standard, uh, you know, CMOS transistors. Um, but, uh, in, in this kind of paradigm of computing, uh, stochastic digital circuits, we encode numbers in, uh, random bits,

uh, bits that are Bernoulli random variables like coin flips. Um, and the probability that the bit is one is kind of encoding a number. And so different numbers are encoded by random bits with different probabilities. Um, and so you can imagine that using this way of like, of formatting or encoding numbers is more robust to errors than, um, it is if, if we just formatted all our numbers in binary, uh, in, if I have a binary number, if I accidentally flipped, uh, you know, the least significant bit that causes us pretty small error. But if I accidentally flipped, uh, the most significant bit that creates a potentially like a catastrophic error. And so that's kind of the reason that standard like binary encoding is not the most resilient to error, uh, to like bit flip errors.
And so when we use instead, uh, stochastic digital circuits, uh, because our processing is resilient to bit flip errors, we can, in some sense, like cut corners with the design and not build in as many guard rails that, uh, prevent bit flip errors. Uh, and one example of that would be, um, well, I mentioned earlier how synchronized, like global centralized synchronization is often assumed in, uh, integrated circuits. But if you have some resilience against bit flip errors, you can often, uh, use some kind of trick so that you don't have to have a global, uh, clock that everything is synchronized to, or instead you could have, you know, many local clocks and, and each, you know, different parts of the chip are just kind of locally synchronized, uh, but not perfectly. Those local clocks can drift or skew a little bit relative to each other, which is called polysynchronous clocking. Um, and, and using methods like that, that take advantage of the error resilience of this paradigm, uh, can allow you to, uh, you know, drive your circuit faster or to, uh, use less energy, basically. Um, and then of course there's, uh, approach, there's other kind of physics-based approaches that, uh, like optical, optical computing or, uh, you know, integrated photonics, for example, which I won't get into, uh, very much. Um, but, uh, yeah, some of our coauthors, uh, in particular, you know, like Peter McMahon is more of an expert on that kind of stuff. But, um, I guess in general, I just wanted to emphasize that there are a lot of different kind of building blocks that you could use to build circuits, uh, you know, make chips that, um, start to do away with some of these assumptions. Um, oh, I guess another good example of a computing platform or, you know, device that, uh, does away with one of those assumptions would be memristors, which are, of course, intended to behave statefully. So the, the way a memristor behaves should depend on kind of its history of what voltages have been applied to it before. Uh, so it kind of gets rid of this assumption of statelessness. Uh, and then you could also make like an emulator of a, a memristor, uh, using, you know, transistors and capacitors and so on. Uh, so you can, you can even without having, uh, you know, memristor device per se, you can still build circuits that kind of behave in the same way of behaving statefully. Um, so I've talked about kind of a few different, you know, platforms and building blocks that can be used, um, to make like physics-based ASICs. Uh, and so now, uh, you may be starting to wonder, well, how do we know that, uh, these devices are actually going to be useful? Right? What kind of problems will we be able to solve? You know, even if we build a chip that is, you know, is, isn't, doesn't have statelessness or determinism or unidirectional information flow and so on, uh, what's it going to be good for? Um, so this is where, uh, we need to think about that,

um, like hardware algorithm co-design approach I was talking about. Uh, so we, we have some device which has some, you know, physical processes that occur on it. Uh, and we think of that as, as kind of this circle over here in this Venn diagram, um, or, and as we put it here is like what physics wants to compute. Uh, and because when you have like some, um, for example, some network of electrical devices like, you know, transistors and capacitors and so on, uh, there's going to be some, you know, some mathematical function that relates its inputs to its outputs, you know, potentially some complicated function, uh, which includes a lot of, you know, physics that happens. Uh, but we don't know that that's useful per se. So that's kind of what the physics wants to compute. And there's also like what, uh, functions are useful in our algorithms, uh, what, you know, what are kind of the primitive building blocks of our algorithms. Um, and so what we want to do is we want to find like the overlap of these two sets basically, and find the operations that are performed efficiently on a device due to the physics of how the device works and are also useful in, uh, you know, in some algorithm basically. Um, so that's kind of the idea of hardware algorithm co-design. Um, and so then I guess I'll talk a little bit about like examples of how that might work. Um, so I think like one good example of a class of algorithms where we would expect these like physics-based ASICs to perform well is in, uh, well, in machine learning in general, where you have a function that is parameterized by a set of like learned weights. Um, because, uh, one thing that's useful about this is that a, you know, a neural network is adaptable in some way. And that if you make some change to its activation function, for example, and then retrain it, uh, it will, you know, often, you know, in general be able to modify the weights so that it still computes the, the correct function, uh, but just does it in a slightly different way because now it, you've changed the activation functions and also changed the weights to somehow compensate for that. So, um, similarly when, if we have some physics-based ASIC that has, uh, some kind of intricate physical processes that are used to evaluate functions, uh, we don't necessarily get to choose what those functions are. Um, they might be, you know, very complicated functions, uh, and they're not necessarily, you know, immediately useful, but if we use, if we, um, combine those functions that are natural to perform on the hardware with a learning algorithm and trainable weights, then in some sense, we expect the, uh, the model to be able to adapt to the, um, the functions that it is given, basically. So you can kind of think of that as analogous to the idea of training a model for some, you know, different, for, for different kinds of activation functions, and you could also train a model for, uh, that can take advantage of the physics on a, you know, of the operations that are naturally performed on some physics-based ASIC. Um, so that's kind of one general class is like training, uh, or in running, you know, training and inference for neural networks on physics-based ASICs. Um, in fact, a lot of the, uh, you know, hardware paradigms that, uh, we've talked about are being used or, or, or have been proposed to be used for exactly that. Uh, so we have this area of like training and inference for neural networks. Um, another good example is, uh, like scientific computing. So a lot of the time, you know, some of the most demanding, um, computations that are done are simulating actual physical systems. So as one example, you could think of like, uh, molecular dynamics, uh, where you have some large, uh, molecule, which is basically a network of atoms

that are, you know, connected to each other, you know, of course by, you know, bonds and so on. Um, and then there are the laws of motion that govern, you know, how all those atoms move, right? And in general, like, you know, the way that people report kind of performance for molecular dynamics simulation, I believe is, is often quoted in like nanoseconds per day. Like how long do you have to run your, uh, or like how many days do you have to run your simulation to simulate a molecule evolving for like a nanosecond is a relevant question in molecular dynamics. And so that kind of goes to show that somehow the physical system itself, the molecule is much more efficient at running its own dynamics than the, uh, you know, then, uh, digital computer is. And so then we could ask, well, is there some circuit that behaves more like the actual molecule physically, or that has like similar equations of motion to a molecule? Uh, that would be a lucky coincidence, right? If we could find a circuit that has very similar equations of motion to some molecule and could use it to simulate the molecular dynamics or, you know, more generally, um, we can look at some of the properties of the physical system in, you know, that's, that's there in molecular dynamics and say, well, can we, um, do we really need all of these properties in our circuit that we talked about before, like, you know, statelessness and unit directionality, et cetera, uh, in order to build some, you know, build some algorithm for simulating the molecule. Uh, and that's kind of a more general approach of like, we're not necessarily looking for an exact mapping between the dynamics of the circuit and the dynamics of the molecule, but we're kind of looking at a more abstract level, like what are the properties of the physical system we're trying to simulate? What are the properties of the circuit? And is there some, uh, kind of, um, agreement between those in some way? Uh, so as a good example would be, uh, determinism, uh, in molecular dynamics simulations, of course, there's, there's generally noise that's added, uh, which represents the influence of the heat bath. Uh, you, you offer, well, you'll simulate a molecule, not just in a vacuum, but typically in some, you know, solvent. And, uh, that solvent could be simulated in different ways, uh, either, you know, by explicitly simulating a bunch of water molecules or by adding a kind of noise terms in to, uh, simulate the effects of random impacts from water molecules or et cetera. Uh, and because we're running an algorithm where in some way there's like random behavior, uh, which again, modeling the heat bath, uh, we could, we start to wonder if, if we need to make, if it's actually necessary for our circuit to behave in a totally deterministic way, or if it's okay if our circuit has, uh, random errors at some rate. Uh, so I think that's a good example of kind of connecting the properties of the circuit we're using for the confusion for computing to the application that's required by some like scientific computing task. Um, and, and there may be, uh, you know, other ways to do this as well. You know, for example, if I have a molecule, the, there, um, there isn't necessarily like, uh, kind of something that behaves like a global clock that's, you know, talking to every single atom and synchronizing their motions, right? Only, you know, there's only kind of local, uh, interaction and local connectivity in, in typically in, in molecules. And so there might be a way of using a, um, a computing architecture that doesn't have a global clock or has something more like that polysynchronous clocking I was talking about, uh, and maps onto the molecular dynamics problem somehow. And, and similar kind of arguments could

apply for these, these other properties as well, even if the details of how that would work isn't, uh, exactly worked out. Um, and so in general, uh, you know, as, as we were talking about earlier, when we kind of get rid of those idealized properties, we are allowing our circuit to behave more like a generic physical system that we might find in nature. And so it might end up that it's well suited to simulating physical systems that occur in nature, uh, like, like molecules, for example. Um, so another kind of class of algorithms where we expect like physics-based ASICs to be useful is, um, well, one thing in randomized algorithms in general, uh, because I was talking about how, um, well, if we don't require our circuit to be deterministic, we can cut some corners and make it more energy efficient and fast. And so in a lot of algorithms that actually require randomness, like, uh, MCMC, for example, or, or, uh, like sampling and, you know, Bayesian inference, um, there, you know, the algorithm is naturally resilient to errors because, uh, randomness is required by the algorithm. And so we expect, you know, physics-based ASICs that get rid of properties like determinism to be, uh, well suited to these kinds of algorithms. Uh, and, and by a similar, a very similar argument in a lot of, you know, generative AI, like, uh, you know, image generation, video generation, materials discovery, which is, you know, for an example, like generating new molecules, uh, often these are solved using, like, diffusion models, which are also like randomized algorithms because they're basically simulations of a stochastic differential equation, uh, which is actually similar to the, you know, molecular dynamics that the way that we model a molecule evolving under a heat bath is like a differential equation that, that is, uh, based on all of the forces, uh, between atoms and Newton's laws. And then we add noise, uh, and similarly with the diffusion model, the way that we sample an image is by running a stochastic differential equation, you know, which is, has some drift velocity and then some noise term. Uh, so in some sense there, there's even a similarity between, you know, diffusion models and, and molecular dynamics at some, like, algorithmic level. Uh, and then that's actually one of the applications we're, uh, very interested in, I would say, is, is diffusion models in particular because that's, uh, rapidly growing in kind of its usefulness, uh, and its, like, computational cost. Um, and then there's also some work, uh, on, we'll see, you know, we, we had an earlier paper, thermodynamic linear algebra, where we talked about how you could use a, a kind of thermodynamic, uh, processor to solve problems in linear algebra, like, linear systems of equations or inverting matrices, matrix exponentials. Um, and if you're interested in that, you can check out the paper, uh, thermodynamic linear algebra. And then, um, there's, yeah, our other paper that I actually talked about last time I was on this, uh, um, was the, uh, thermodynamic Bayesian inference, where we talked about how to use a thermodynamic processing device for, uh, like sampling from Bayesian posteriors. Uh, so those were some examples of, like, different applications area, different application areas. And I tried to give a little bit of intuition for like why we think different application areas might map well to like physics based ASICs and more kind of, I guess, even more broadly, like why we expect that general purpose digital architectures are kind of overbuilt, uh, for a lot of these applications, or at least they are built to at great expense to guarantee certain properties, which are not needed for, for many of

these applications. Um, so I think that's, you know, basically what I wanted to say about, um, kind of what physics based ASICs are and, uh, you know, how we're thinking about algorithm co-design and what, what we think they're going to be useful for. I guess we also talked a little about in this paper about, uh, kind of where we see the field going and what are some of the next steps, I guess, in this cause to action. So, uh, I'll say a few words about that. Um, so for one thing, uh, there, there are going to be, uh, you know, algorithms that the GPU is very good at, um, and are probably not going to benefit from, or not, or at least not going to benefit as much from, you know, very new unconventional computing paradigms. Uh, but there's also going to be some applications where that are not, you know, very naturally suited to GPUs. Uh, and so that's one of the most, or yeah, one of the first and fundamental things, uh, to do in our calls to action is to kind of identify the set of applications that don't map well to GPUs or that GPUs aren't well optimized for, um, and kind of identify, you know, why they are not well optimized, uh, why the GPU isn't well optimized for those applications. Um, and how, you know, have a good understanding of that. Uh, so, so that's one kind of, uh, call to action. And then another one, uh, of course is actually like co-designing the algorithms and hardware, uh, for physics based ASICs. Uh, so yeah, and we give the example of here of how like transformers have been designed, you know, to run well on GPUs. Um, and so similarly, like what, what's kind of the equivalent of the transformer for, um, uh, for physics based ASICs. Well, you know, maybe we don't know yet. Uh, maybe it's these, uh, energy based transformers we're looking at. Well, either way, I think there's a lot of room to, um, you know, explore new algorithms and hardware, uh, that will, you know, have a good agreement between the algorithm and physics based ASICs, uh, and can be used for, you know, AI. Um, and then, uh, well, there's going to be, yeah. So we talked about like developing the full stack for physics based ASICs. So there's basically just a lot of new kinds of layers of software that are going to be needed, uh, like simulation, uh, of, so yeah, like first, like how can we, can we use like a GPU to simulate how our physics based ASIC would perform, uh, so that we can kind of solve design problems before we actually build it. Uh, and can we take, you know, existing machine learning algorithms and compile them so that they'll run efficiently using our physical primitives on the physics based ASIC. That's another thing, like kind of a compilation layer. Uh, so that would be, you know, a significant amount of work. Um, and then I, I think we also have a note about like, yeah, explaining goals, uh, you know, in a way explaining the field basically in a way that's easy to understand for people in different, uh, from different backgrounds, like electrical engineering or computer science and so on. Uh, just because this is a very like interdisciplinary field at the moment. Um, which is one of the things that's so, uh, exciting and fun about it, I think is that it's very, very truly like interdisciplinary and combining physics and, you know, uh, engineering, circuit design and theoretical computer science in a very, in math and kind of very interesting combination. Um, so yeah, I think that that covers everything I wanted to say about the paper. And, um, I guess at this point I'll take any, any questions if there were any other questions. Awesome. Awesome. All right. Okay.

Here's where that took me the three uncertainties. What is a unifying framework for cognitive science? What is a unifying framework for computer science? And how are those questions related, especially in the context of cognition as inference slash compute as cognition? Like what are those sort of domain or discipline scale syntheses in the cognitive science and the computer science department, so to speak? And what does that look like as we increasingly see those two used by and forward with each other? So if you have any thoughts, go forward, but that was just my, where it took me. Interesting. Okay. So, right. Okay. So you're asking like, what is a unifying framework for cognitive science or, and, or yeah, and I guess a unifying framework for computer science. And are you also kind of like, like, uh, like gesturing at like a unifying framework that would unify both of those unifying frameworks as well? Kind of? Or?

Yeah. It's like all these branches coming together that you brought together with the authorship and the perspectives in this paper. And then over here in cognitive sciences and active inference, that's the quest of the unifying cognitive model that attention, memory, anticipation, all these different cognitive phenomena would be approached in a unifying handling. And we're using those probabilistic compute tools along the way and even interpreting what the materials, classical computers and unconventional computers, what they do as this kind of like material intelligence. So it is getting very hot and heavy in that area.

Yeah. I see. Yeah. No, I definitely like, yeah, thinking about things like that as well. I would say that I haven't explored as much like the, uh, kind of interactions with the cognitive side, you know, cognitive science side of things. Although, although I know there's a lot of kind of rich, interesting things to think about there, uh, as like, you know, for example, like some, some, uh, you know, things in this paradigm people do are things like spiking neural networks and things like that, that are at least inspired by and arguably, you know, behaving in a similar way, perhaps to think, you know, things in the brain. Um, and, and that was also something we thought about a little bit in our, in our, you know, thermo Bayesian inference paper was, uh, can this, can this somehow be mapped onto ideas about like how, you know, people reason, uh, under, or, you know, how, yeah, we, yeah, how people reason under uncertainty, things like that. Um, so, yeah, I think there probably is a lot to explore there. Also, yeah, just goes to show just how kind of like interdisciplinary this field is. Um, another thing I think about in terms of like unification, which is, is a little bit, uh, different from that, but may, you know, have some overlaps is like, unified is like, uh, kind of unifying computer science, or at least, uh, computational complexity theory with, uh, thermodynamics in some way, which I think is a kind of interesting idea. I'm not sure like how realistic it is at this point, but, um, there are, uh, basically the reason I think that there is going to be some overlap there and perhaps like some unification is that these are like kind of some of the two, two important theories that give us estimates of how much, uh, you know, basically how much work it takes to do something. Uh, so computational complexity theory gives us estimates of, you know, how does the amount of time, uh, or you could also think of it as like, or, well, the amount of space or even the amount of energy required scale with, uh, the size of your computation that you have to do basically. And

similarly in thermodynamics, you can derive results that tell you how much like physical work you have to do, uh, in, meaning like literally the amount of energy, uh, that you spend or dissipate in order to, uh, perform some, uh, you know, perform some change in the system state. And there already are some small like overlaps where this becomes more explicit, like of course, Landauer's principle and similar like generalizations of Landauer's principle that, uh, make a link between thermodynamic work and computational operations. But in general, I think that there's a larger kind of like program that could be pursued of, um, mapping statements and computational complexity theory somehow into thermodynamic statements about how much work is required to bring about some change in state of the system basically. Uh, so I think that's kind of another, and that could even perhaps overwrap with like the idea you were mentioning of like unifying, uh, kind of these not, you know, exotic computing architectures with, with cognitive science and stuff like that. So yeah, I think that's kind of an exciting idea. Cool. I'll ask one more question, then read the questions that people have added in the chat. So how do you see biology and biological materials in the scheme? Figure three seems to be mainly abiotic focused. Uh, figure three. Um, sorry, what was the last phrase you said? It was, it was something focused? Abiotic. Abiotic? Where's that? Sorry, what does that mean? These, uh, are not arising from a living system. They're machines or engineered, though there's some analogy with biological systems, but just how did that
come up or where do you see biological processes and materials generation in this scheme that the paper lays out? I see. Okay. Yeah. Okay. So I'm going to kind of, I'm going to venture a little bit outside of my, uh, wheelhouse here because I, I'm not, uh, you know, an expert in biology by any means. Um, and so that's why I'm a little bit more comfortable kind of drawing analogies between the behaviors of these circuits and like generic dynamical systems, like a molecule, for example. Uh, but of course, uh, you know, organisms are also made out of molecules. Um, and even at a higher level of, you know, even at a higher structural level organisms might have some similarities, uh, you know, in, in terms of those properties I was mentioning before, I guess, um, um, in general, yeah, I guess, yeah, one example might be good was that Kuramoto model thing I was just talking about. Uh, so there, I've seen that there's some work on using the Kuramoto model, which is a model of how a bunch of different oscillators can synchronize with each other to model how, um, synchronization, uh, can occur in the brain, uh, which I think has to do with, uh, you know, brain waves that are, that are, you know, happening with different frequencies, although I don't know much about that. Um, but I, I think that general, like the, and I'm not sure if people know whether the Kuramoto model is actually a good, uh, or whether it's a well-justified model of that, but at any rate, the Kuramoto model is a decentralized model of synchronization, where you have a bunch of things that have, you know, have pairwise interactions talking to each other instead of one global clock that's coordinating everything. And so there we see, uh, a physics-based ASIC that kind of gets, gets rid of assumption four might be more similar in architecture to a biological system than it is to something like a GPU. Um, so I think that that's like one way of answering your question is that I would expect that physics-based ASICs may have more

similarities in architecture to biological systems being, uh, you know, decentralized. And, uh, although I don't, I don't really know enough about, you know, kind of typical biological systems to probably give a very good answer, but, uh, yeah, I think that's one thought I have on it. Cool. Okay. From the live chat, Gorto68 wrote, Cubism is my jam. That was just a comment. And then a question. Max, have you noticed a trend in the last eight years in a new physics of geometry and ultra high frequency applications to metamaterials and nanomachines? Um, let's see. Okay. Try the last eight years of, um, metamaterials and nanomachines. Wait, what was the first part? Something relating to nanomaterials and metamaterials? New physics of geometry and ultra high frequency applications to metamaterials and nanomachines.

I would say I have not noticed that, but I haven't been very focused on those fields. Uh, so yeah, I haven't noticed it, but that doesn't mean it isn't happening. Uh, but yeah, it sounds, sounds interesting. Yeah. What has changed over the last eight years or months or days? Why is this coming to fruition in your research and development at this time? That's a good question. So one thing that's changed, I guess, and yeah, let's say the last decade or something is that, uh, quantum computing turned out to be like harder than a lot of people thought it was going to be, I think. Um, and, uh, so I, I remember like early on when I was beginning grad school, uh, there were a lot of people, uh, who were, and I was, well, not quite well, I was kind of, I was kind of working more on the theory, I guess, of like thermodynamics of quantum computers. But there were a lot of people at that time who were working on like quantum compilers, uh, things like that, that they were supposed to make the, they were supposed to solve the problems of like software engineering for a quantum computer. And that just kind of goes to show how optimistic people were at that point, I think, because people already kind of treated the engineering challenge of building a quantum computer as like a foregone conclusion that would happen in, you know, the next decade or something. And then we would run into this problem of needing to be able to design scalable software for it. But as you know, as far as I know, it, it didn't quite turn out that way. And the problem is still, uh, we're still stuck with the problem of actually building, you know, the large, uh, quantum computers. And so now people are thinking more about like error correction codes and things like that, that, uh, that, you know, will basically allow us to scale them more easily. But I guess that that's one reason why, at least in the story of how I became interested in these kinds of approaches to computing, that these are unconventional, classical approaches to computing that, you know, don't have their, well, they, they still, you know, have some scalability and engineering challenges associated with them. They're not the same kinds of challenges that, uh, were so hard and still are so hard in quantum computing, I guess. Uh, that would be one answer. Uh, and the other answer, I guess, is, uh, is kind of related to what I was talking about with the end of Denard scaling, although that happened, you know, more like two decades ago. Um, but arguably it's now is when it started, it's kind of being acutely, most acutely felt, I guess, uh, the end of Denard scaling, because we're finally getting to the point where, um, the energy costs are just kind of accelerating massively, uh, due to our, you know, rapidly expanding compute needed for AI. So those are kind of the two things I would point to, I guess.

Cool. Yeah. Where did the paper take you all and the authors? Like, where do you, where do you go from here? What, what milestones are you looking for next?

Right. Well, yeah. So, so at normal computing, uh, you know, we're making our own chips. Um, and so we have, you know, milestones for those, for the development of those chips, uh, which have to do with like specific AI, generative AI algorithms that we want to run on them. Uh, so that involves like simulating how the chips are going to perform and, and getting out kind of estimates of their performance from our simulations, uh, building the chips. We've already made, uh, a lot of progress on, on that. Um, and, uh, and then kind of thinking through like how these chips are actually going to be used like commercially. Uh, so that's what I, when I, uh, think about kind of like my piece of this puzzle of, um, physics-based ASICs, it really has to do with our, uh, stochastic digital kind of thermodynamic stochastic digital processors we're making at, uh, at normal computing to solve some of these problems in AI.

Very cool. Uh, I guess one sort of last question, and then feel free to make any comments. Like, what is the educational trajectory? How do people learn up on this? Is it going to look like type theory and programming languages and computer science undergrad, or what kinds of background are useful to be learning and getting familiar with to kind of skate to where the puck is going?

Right. So I would say that for one thing, uh, you know, definitely, uh, people experienced in, you know, silicon engineering, uh, you know, if, if people who have a skill set in designing conventional ASICs and have a deep knowledge of, you know, of integrated circuits from that, well, you know, that knowledge will absolutely, I think, transfer over to, um, kind of more unconventional architectures. And it might be even more, uh, might be even more fun to think about because you, it's kind of throwing away some of the rules that are, uh, it's kind of like you, you know, solving some of the same kinds of problems, but where many of the rules have been thrown away, I guess. So it becomes kind of a different game. Uh, so that's one background, one way of getting there. Uh, another, which is more, which is the way that I kind of got here is the, uh, kind of more, uh, theoretical physics and in particular, like statistical physics and thermodynamics end up being very relevant, uh, especially because a lot of algorithms and AI have some algorithms and AI have been designed, uh, inspired by, or based on, uh, uh, ideas in statistical physics. So like, for example, diffusion models, um, or more generally in like machine learning, uh, there's like Bayesian inference models and Monte Carlo methods that are deeply, deeply related to, uh, things in statistical physics. Uh, and so in general, like having both knowledge of the algorithms and of the physical pictures that those algorithms map onto kind of puts you in, I think a good position to think about like what might the ideal piece of hardware look like to run this algorithm, uh, because you can kind of think about like what physical models map onto an algorithm, I guess. So that's another background that's, that I think is a good kind of route to this, uh, this way of thinking is like statistical physics, uh, et cetera. Um, and then of course, uh, people with like, yeah, people with math, math backgrounds, uh, you know, math, statistics, uh, computer science, um, and AI in general, uh, are of course all important. And especially, uh, when we get into things like, um, performance bottlenecks, like, uh, knowing,

you know, knowing the answers to the questions, like what's the most costly part of running this, uh, pipeline, if I'm running a diffusion model or I'm running a large language model, where is the majority of the computation being done, uh, which is kind of related to like theoretical computer science and complexity theory, but isn't quite the same. It's more of like a practical question of like, where are the actual bottlenecks in practice? Um, so, and, and so, you know, that would be the skillset more of people who do like, you know, large scale machine learning, infrastructure, or training and inference and stuff like that. Um, and so I think all those like skillsets are, you know, people, uh, are in a very good position to kind of move into this field. I do hope that more and more people do start to get involved in this. Yeah. Cool. I'm going to read a very interesting related comment, but then feel free to make any last additions there. Okay. Talal Zahid wrote, how digital education will advance with AI systems running on special chips, ASICs using probabilistic models to personalize learning over time as digital education platforms scale globally, delivering personalized and adaptive learning, which requires low latency AI computation. This research explores the deployment of stochastic AI models on ASICs to achieve real-time adaptation and user learning paths, reducing energy consumption while maximizing student outcomes. I see that, that does sound like really interesting research. And what I like about it is it's like, it's an example of like kind of connecting the full stack and end in the sense that like it talks about the end application. What are some properties of that application, like requiring low latency and adaptive behavior, although I'm not, I, you know, I haven't read up very much on the application, but it's, it sounds plausible that those are requirements. And then connecting that to like, what algorithms we're going to use for it. And in particular, like, you know, large language models or, or, uh, Bayesian, uh, reasoning models, and then like what hardware would be required to accelerate those algorithms. So kind of connecting the, all the way from the application to the algorithms, to the hardware is exactly the right way of thinking in order to do something meaningful, I think. So I really liked that comment. Yeah. Cool. And yes, closing the loop with the improvements and the improving improvements. Uh, any other comments? Um, no, I think, yeah, I think I covered everything. Uh, and yeah, just again, I really enjoyed coming on. Uh, so yeah, always, always a pleasure. Uh, and yeah, thanks for having me on. Cool. Looking forward to the next episode. All right. Sounds good. Thanks. See you later.
All right.

Thank you.